Listing 6.23 Fetching configurations for specific indexes

```
GET mov*,stu*
```

Although all these `GET` commands fetch the aliases, mappings, and settings of the specified indexes, there is yet another way to return that information. Say we want to fetch an individual configuration for a specific index. We can use the relevant APIs to do so.

Listing 6.24 Getting individual configurations for an index

```
GET cars/_settings
GET cars/_mapping
GET cars/_alias
```

Gets the settings from the _settings endpoint

Gets the mappings from the _mapping endpoint

Gets the aliases from the _alias endpoint

These `GET` commands return the specified configurations for a public index. Let's look at retrieving this information for a hidden index next.

> **NOTE** To determine whether the index exists in the cluster, we can use the command `HEAD <index_name>`. For example, `HEAD cars` returns the error code `200–OK` if the index exists and the error `404–Not Found` if the index doesn't exist.

6.3.2 Reading hidden indexes

As mentioned, there are two types of indexes: the normal (public) indexes that we've worked with so far and hidden indexes. Similar to hidden folders in computer file systems, which are prefixed with a dot (.etc, .git, .bash, etc.), hidden indexes are usually reserved for system management. They are used for Kibana, application health, and so forth. For example we can create a hidden index by executing the command `PUT .old_cars` (note the dot in front of the index name).

> **NOTE** Although we can use hidden indexes mimicking public indexes for operational use (because Elasticsearch has no checks and balances in version 8 and earlier), this will change in future versions. All hidden indexes will be reserved for system-related work. With this change in mind, my advice is to be mindful of creating hidden indexes for business-related data.

The `GET _all` or `GET *` call fetches all indexes, including hidden indexes. For example, figure 6.5 shows the result of issuing a `GET _all` command: it returns the entire index list, including hidden indexes (indexes with a dot in front of the name).

Now that we know how to create and read indexes, we can, if the need arises, execute delete operations on them. This is discussed in detail in the next section.